

Plano de Projeto

Finanças SaaS — Gestão Financeira Pessoal com Dashboard

Documento de referência do projeto. Toda decisão de escopo, arquitetura e prioridade tomada daqui em diante deve ser checada contra este plano. Atualizar este documento sempre que o escopo mudar.

Versão 1.0 — Setembro de 2026 · Projeto solo

1. Visão Geral do Projeto

O que é: um SaaS de gestão financeira pessoal. O usuário importa o extrato bancário em CSV, o sistema categoriza os gastos automaticamente e apresenta um dashboard com gráficos de gastos por categoria e evolução mensal.

Por que este projeto: desenvolvido por uma única pessoa, em paralelo à faculdade, com dois objetivos centrais: (1) construir um projeto de portfólio relevante para a área de dados (processamento de dados reais, categorização, agregações, visualização) e (2) desenvolver habilidades de programação fullstack (API, banco de dados, frontend).

Contexto de trabalho: projeto solo, tempo limitado, com uso ativo de IA como copiloto de desenvolvimento — gerando boilerplate e tirando dúvidas, mas sempre com revisão e entendimento do código por parte do desenvolvedor.

2. Objetivos

2.1 Objetivo principal

Ter um produto funcional, simples e bem estruturado, que demonstre domínio do ciclo completo: ingestão de dados → processamento/regra de negócio → persistência → visualização.

2.2 Objetivos de aprendizado

- Processamento de arquivos (parsing de CSV com dados do mundo real, inconsistentes)
- Lógica de negócio (regras de categorização, agregações)
- Visualização de dados (gráficos, dashboards)
- Fullstack: API REST (FastAPI), frontend (React + TypeScript), banco relacional
- Fluxo de trabalho profissional com Git (branches, commits organizados)
- Uso produtivo de IA como ferramenta de desenvolvimento do dia a dia

3. Escopo do Projeto

3.1 Dentro do escopo (v1)

- Upload de extrato em CSV (formato genérico definido no projeto)
- Categorização automática por regras de palavra-chave
- Edição manual de categoria (correção pelo usuário)
- Dashboard com gráfico de gastos por categoria e evolução mensal

- Cadastro de categorias com palavras-chave associadas

3.2 Fora do escopo (v1) — decisão deliberada

Para manter o projeto viável dentro do tempo disponível, os itens abaixo são **propositalmente adiados**. Evitar a tentação de adicioná-los cedo demais:

- Autenticação/múltiplos usuários (fase 2)
- Suporte a múltiplos formatos de CSV de bancos diferentes (fase 3)
- Categorização com Machine Learning (possível evolução futura, não v1)
- Deploy público / infraestrutura em produção (fase 3)
- Exportação de relatórios (fase 3)

4. Requisitos Funcionais

ID	Requisito	Descrição
RF01	Importar CSV	O usuário deve poder enviar um arquivo .csv com colunas data, descricao e valor.
RF02	Validar CSV	O sistema deve validar colunas obrigatórias e formatos, retornando erro claro se inválido.
RF03	Categorizar automaticamente	Ao importar, cada transação deve ser comparada com as keywords das categorias cadastradas.
RF04	Categorizar manualmente	O usuário deve poder alterar a categoria de qualquer transação.
RF05	Cadastrar categorias	O usuário deve poder criar categorias com nome e lista de palavras-chave.
RF06	Listar transações	O usuário deve poder visualizar as transações importadas, com filtro por mês/ano/categoria.
RF07	Dashboard por categoria	O sistema deve exibir gráfico com soma de gastos agrupada por categoria.
RF08	Dashboard mensal	O sistema deve exibir gráfico com evolução de gastos ao longo dos meses.

5. Requisitos Não Funcionais

ID	Requisito	Descrição
RNF01	Simplicidade	Stack e arquitetura devem priorizar simplicidade de manutenção por 1 dev part-time.
RNF02	Documentação de API	A API deve expor documentação automática (Swagger/OpenAPI) para testes sem frontend.
RNF03	Portabilidade de banco	Trocar SQLite por PostgreSQL deve exigir apenas mudança de configuração, não de código.
RNF04	Tipagem	Backend e frontend devem usar tipagem (Pydantic / TypeScript) para reduzir erros e facilitar revisão de código gerado por IA.
RNF05	Versionamento	Todo o código deve estar versionado em Git, seguindo o fluxo de branches definido na seção 8.

6. Stack Tecnológica

Camada	Tecnologia	Motivo
Backend	Python + FastAPI	Tipagem via Pydantic, docs automáticas, integração natural com pandas.
Processamento de dados	pandas	Parsing e manipulação de CSV; biblioteca padrão de mercado em dados.
Banco de dados	SQLite (v1) → PostgreSQL (futuro)	SQLite não exige setup de servidor; migração facilitada via SQLAlchemy.
ORM	SQLAlchemy	Abstrai o banco, facilita troca de SQLite para Postgres.
Frontend	React + TypeScript + Vite	Aprendizado fullstack real; tipagem ajuda a revisar código gerado por IA.
Gráficos	Recharts	Integração simples com componentes React.
Requisições HTTP	axios	Cliente HTTP simples para consumir a API no frontend.

7. Arquitetura

7.1 Estrutura de pastas

```
financas-saas/
|-- backend/
|   |-- app/
|   |   |-- main.py
|   |   |-- models/           (SQLAlchemy: Transaction, Category)
|   |   |-- schemas/         (Pydantic: validacao/serializacao)
|   |   |-- routers/         (upload, transactions, categories, dashboard)
|   |   |-- services/         (csv_parser.py, categorizer.py)
|   |   |-- db.py
|   |   |-- config.py
|   |-- requirements.txt
|-- frontend/
|   |-- src/
|   |   |-- components/      (UploadCSV, TransactionTable, charts/)
|   |   |-- pages/           (Dashboard)
|   |   |-- services/        (api.ts)
|   |   |-- types/           (transaction.ts)
```

7.2 Modelo de dados

Tabela	Campo	Tipo	Observação
categories	id	int (PK)	
categories	name	string	único
categories	keywords	string	palavras-chave separadas por vírgula
transactions	id	int (PK)	
transactions	date	date	
transactions	description	string	descrição bruta do CSV
transactions	amount	decimal	negativo = saída, positivo = entrada

Tabela	Campo	Tipo	Observação
transactions	category_id	int (FK, nullable)	nulo até categorização
transactions	source_file	string	rastreabilidade do CSV de origem
transactions	created_at	datetime	

7.3 Endpoints da API (v1)

Método	Rota	Descrição
POST	/upload	Recebe CSV, faz parsing, categoriza e salva as transações.
GET	/transactions	Lista transações, com filtros de mês/ano/categoria.
PATCH	/transactions/{id}	Atualiza a categoria de uma transação.
GET	/categories	Lista categorias cadastradas.
POST	/categories	Cria uma nova categoria com palavras-chave.
GET	/dashboard/by-category	Retorna soma de gastos agrupada por categoria.
GET	/dashboard/monthly	Retorna evolução de gastos por mês.

8. Fluxo de Trabalho com Git

Fluxo de branches adotado para manter a **main** sempre estável:

- **main** — código estável, testado. Só recebe merge vindo da dev.
- **dev** — branch de integração. Recebe merge das branches de feature já testadas.
- **feature/nome-da-feature** — uma branch por funcionalidade, criada a partir da dev.
- **fix/nome-do-bug** — para correções pontuais.
- **chore/descricao** — para manutenção (dependências, configs, etc).

Fluxo padrão para uma nova funcionalidade:

```
git checkout dev
git checkout -b feature/nome-da-feature
# ... desenvolve e commita ...
git push -u origin feature/nome-da-feature
# merge na dev quando testado
git checkout dev && git merge feature/nome-da-feature && git push
# merge na main quando dev estiver estável
git checkout main && git merge dev && git push
```

Recomendação: apagar a branch de feature após o merge, manter commits pequenos e descritivos, e (opcionalmente) abrir Pull Request de feature → dev no GitHub mesmo trabalhando sozinho, como prática de mercado.

9. Roadmap por Fases

Fase 1 — Núcleo (MVP)

Objetivo: ter o ciclo completo funcionando de ponta a ponta.

- Backend: upload de CSV, categorização por regra, endpoints de transações/categorias/dashboard
- Frontend: tela de upload, tabela de transações, 2 gráficos (categoria e evolução mensal)
- Status: esqueleto inicial já criado (backend + frontend rodando localmente)

Fase 2 — Deixar 'de verdade'

- Autenticação básica (login/sessão)
- Edição de categoria diretamente na interface (feedback loop do usuário)
- Suporte a múltiplos extratos/meses importados
- Melhorias de UX no dashboard (filtros, período customizado)

Fase 3 — Ir além

- Suporte a formatos de CSV de diferentes bancos (detecção/normalização de schema)
- Exportação de relatórios (PDF/Excel)
- Deploy público (backend + frontend + banco em produção)
- Possível evolução da categorização para um classificador de ML treinado com correções manuais

10. Próximos Passos Imediatos

- Validar o núcleo da lógica de negócio via Swagger (/docs): criar categorias, subir um CSV de teste, checar categorização automática
- Rodar o frontend e validar upload + visualização dos gráficos ponta a ponta
- Definir e criar as próximas branches de feature a partir da Fase 2 do roadmap
- Manter este documento atualizado conforme decisões de escopo mudarem

Documento vivo — atualizar conforme o projeto evoluir.